Taming Overfitting: Regularisation in Neural Networks

A hands-on tutorial built with a from-scratch NumPy MLP

Code & reproducible notebook: github.com/<SaiAkhil24>/mlp-regularisation-tutorial

There are neural networks that have sufficient capacity to learn their training data completely but fail to learn on new examples. That distance between fitting and generalising is overfitting, and learning to control it is perhaps the most useful skill in the applied machine learning. You are going to learn the three most popular tools you will use most of the time, L2 weight decay, dropout and early stopping, by observing how they operate on this small, purposely noisy problem. Each figure is created by a Multilayer Perceptron (MLP) built from scratch in NumPy, there is no "magic" in the library call.

It is useful to understand the reasons for overfitting. A model has a certain budget of error which is divided into two parts as follows: loosely speaking, bias (error from being too simple to capture the true pattern, which is the source of the error) and variance (error from being too flexible and chasing the random noise in this particular sample). At the low-bias, high-variance end of things you've got a high-capacity network, which will fit pretty much anything, and will fit the noise, if there's no counter-pressure. Regularisation is just that counter-pressure. Does not remove capacity; it biases towards simpler explanations; it biases towards a larger reduction in variance — and a net gain on unseen data (Goodfellow, Bengio and Courville, 2016).

The setup

The task is a two-moons problem (Pedregosa *et al.*, no date) two overlapping crescents that cannot be separated by a straight line (Figure 1). By design, I have only 120 training points, and they are noisy — that's where overfitting hurts! The model consists of a 2→64→64→1 network using ReLU hidden units, sigmoid output, binary cross-entropy and the Adam optimiser (Kingma and Ba, 2017)). It has more than 4500 parameters for 120 points, so it can memorize a lot of.

It is crucial to two preparation steps. Firstly, the inputs are scaled to have zero mean and unit variance to make both features equally relevant during training and to ensure that the optimiser will converge nicely. Second, and this is often something that beginners neglect, the data is divided three ways. The weights are fit to the training set, the validation set is used to tune things like the regularisation strength and the decision of when to stop training and the test set is only ever touched once to give an honest score. The fastest way to making yourself think that a model generalises when it doesn't is to mix these roles.

Figure 1. The two-moons training set — 120 noisy points, not linearly separable.

Seeing overfitting

What that capacity does, figure 2 will show you. The unregularised network (left) is able to get to 100% training accuracy by wrapping its decision boundary around individual points – even creating little islands of accuracy to account for noise – but manages only 0.87 on the test set. The boundary refers to the sample itself and not to the shape underneath. This is the visual signature of overfitting: a decision surface much more complicated than warranted by the data.

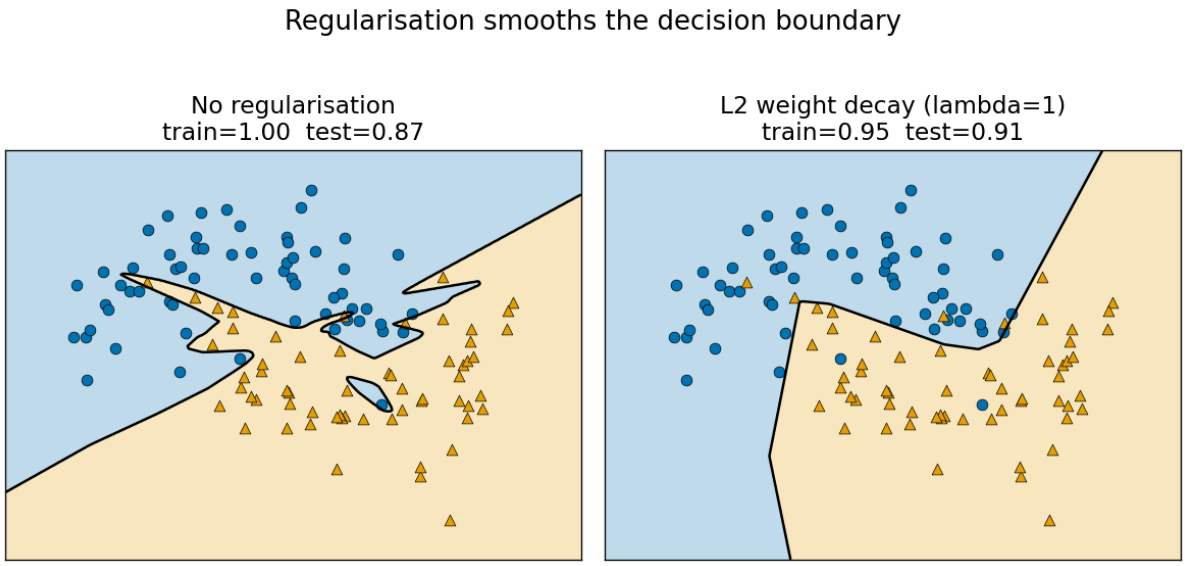

Figure 2. Same network, two outcomes. L2 weight decay (right) smooths the boundary and improves test accuracy.

Before the training accuracy is 100%, this is a failure because of the reason. The model has been trained on the sample, not on the process that produced it. The noise in those 120 points is also unique to this draw; a new draw would have noise in other locations, and a boundary that would work right for one sample would be wrong for the next. A good model would model only the part of the data that would repeat, the crescent shape, and not the part of the data that would not repeat. There is no point to the game.

Even without a 2-D picture, overfitting is easily apparent from the learning curves (Figure 3). If this is not done, the training loss will continue to decrease with the validation loss eventually reaching a minimum and then increasing. As soon as validation loss begins to rise, while training loss continues to fall, you are overfitting — this is the real alarm bell.

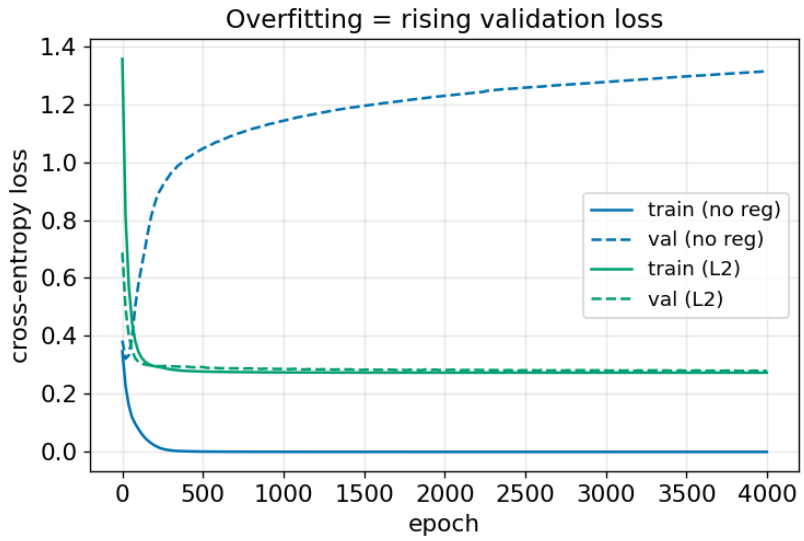

Figure 3. The learning-curve signature of overfitting: a rising validation loss.

L2 weight decay

The first remedy adds a penalty proportional to the squared size of the weights to the loss:

$$L = BCE + (\lambda / 2) \cdot \Sigma \, ||W||^2$$

Weight decay (Krogh and Hertz, 1991): Differentiating each weight gradient adds a term + λW, which means taking the usual gradient step and shrinking the weight by a little bit. Weights will only remain large if the data continually drives them that way. The intuition behind this is that a neural network can only change rapidly at places where its weights are large, so penalising large weights will lead to a smoother function of the network which is not allowed to wrap tightly around any single noisy point.

There is also a clean reading as a probability. Minimising loss and adding an L2 penalty is mathematically equivalent to maximum-a-posteriori estimation with a zero-mean Gaussian distribution over weights — λ represents our prior belief that weights should be small (before observation of data). Regularisation is then no mere random trick, but a statement of prior belief, about the shape of a reasonable solution (Goodfellow *et al.*, 2014).

In Figure 2 (right), the same network trained with λ = 1 discards the islands and finds the crescent boundary again, this time with an accuracy of 0.91 on the test set, but only 0.95 on the training set—a classic example of the training-test accuracy trade-off.

How much regularisation?

What figure 4 does is sweep λ over five orders of magnitude and lay bare the bias–variance trade-off. If the amount is too little, the model overfits (high training, lower test accuracy) and if it is too much, the weights get crushed before they can learn, resulting in both curves approaching 0.5 (chance level). The best generalisation is located in the middle, close to λ = 1. Most important, this curve is read from a validation split, NOT the one you actually report.

There are two habits to this that are practical. When searching for λ it is important to search on a logarithmic grid (…, 0.1, 1, 10, …), since it is the order of magnitude that is important. Also, when data is limited (as in this case), a single validation split is noisy, and it is better to use k-fold cross-validation:

repeatedly cross-validate using all possible k-folds and average the scores and choose the λ that has the best average score. The one most important picture in this tutorial is the shape in Figure 4 - learn to recognize this and you can diagnose almost any model.

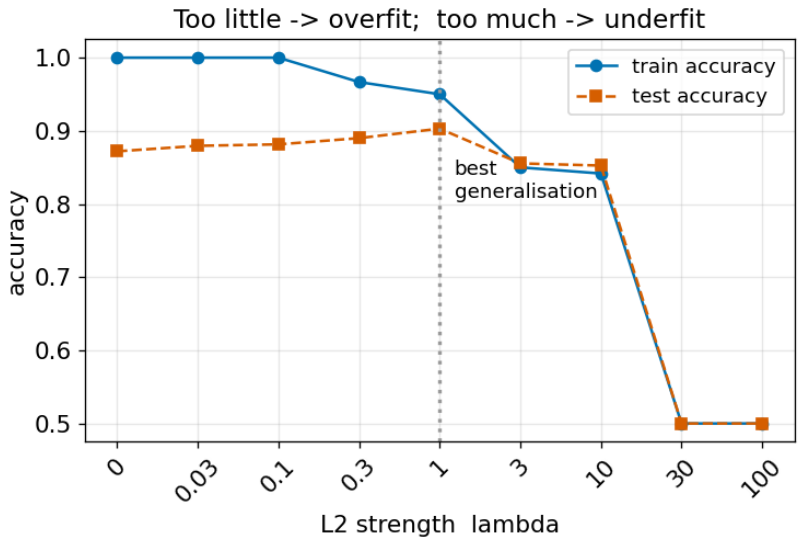

Figure 4. Too little regularisation overfits; too much underfits. The sweet spot is near λ = 1.

Dropout and early stopping

There are two additional tools that complete the kit. Dropout (Srivastava *et al.*, 2014) degrades the network by randomly removing a certain number of hidden units p and dividing the remaining units by (1 − p) (called inverted dropout), ensuring that the expected signal remains the same throughout training. Dropout can be thought of as training a large ensemble of networks, which share weights, at each step, and then averaging them at inference time. No unit ever expects other units to be there so the network learns redundant and robust features. It can be turned on in the code by a single dropout argument. The more the network grows the more it helps — on this small network with 2 features it helps only marginally, but on large, deep networks it is sometimes the regulariser that makes or breaks.

The simplest regulariser is to choose the cheapest one, early stopping (Prechelt, 1998): don't change the loss at all, but just keep the weights from the epoch that gave the lowest validation loss and stop when the validation loss hasn't gone down for a certain number of epochs (the patience). Weights grow gradually during training, stopping early keeps them small – so it works much like weight decay, but free and without needing to tune a λ. That's at around epoch 25 in figure 5, just before the model starts to increase its validation loss.

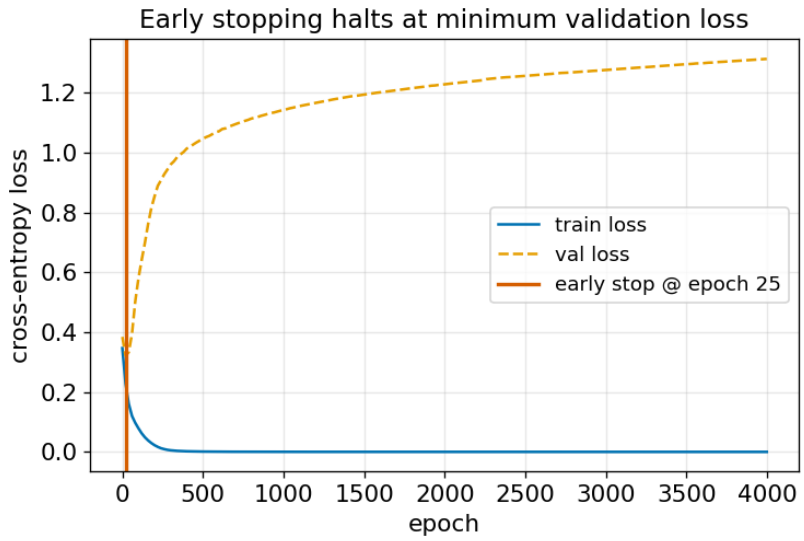

Figure 5. Early stopping halts training at the minimum validation loss.

A practical recipe

1. Split your data into train, validation and test sets before anything else.
2. Train an unregularised model first and plot train vs validation loss to confirm overfitting exists.
3. Add L2 weight decay and tune λ on the validation set with a log-scale sweep (Figure 4).
4. Add dropout when the network is large or the data is scarce.
5. Use early stopping as a safety net regardless.
6. Report results only on the held-out test set, touched once.

Beyond these three

The workhorses, but there's a larger family. The absolute value of weights are being penalised by L1 regularisation, which will push many weights to exactly zero, providing a sparse or feature selecting model. The most reliable remedy to variance is more varied data, and this remedy is often Data augmentation — adding rotated, cropped or noisy copies of the inputs. Batch normalisation is a byproduct of training stability and mildly regularising. The same objective as our 3 is to constrain the model to explain the signal, but not the noise.

Accessibility

All figures are drawn using the Okabe–Ito colour-blind-safe palette and colour is used in combination with different markers and line styles, which makes them easy to read in grey or when someone is colour blind. Descriptive alt text for each figure and the notebook is fully commented.

Conclusion

Regularisation is not a trick that is added on at the end—it is a way of instructing a flexible model to stick to simple explanations. The three tools here attack the same problem from different angles: weight decay shrinks the weights through the loss, dropout introduces randomness to create redundancy and early stopping caps the extent to which the training can wander. The diagnostic transferable skill is: Partition your data into the training set and the validation set honestly, plot validation loss against

training loss, and look for the signature of overfitting and the U-shaped curve of generalisation, and hit the right control. Once that loop is mastered, networks can be trained to be much more capable without fear of something quietly memorizing its way to an unhelpful solution.

References

Goodfellow, I.J. *et al.* (2014) “Generative Adversarial Networks.” arXiv. Available at: https://doi.org/10.48550/arXiv.1406.2661.

Kingma, D.P. and Ba, J. (2017) “Adam: A Method for Stochastic Optimization.” arXiv. Available at: https://doi.org/10.48550/arXiv.1412.6980.

Krogh, A. and Hertz, J. (1991) “A Simple Weight Decay Can Improve Generalization,” *Advances in Neural Information Processing Systems*. Morgan-Kaufmann. Available at: https://proceedings.neurips.cc/paper/1991/hash/8eefcfdf5990e441f0fb6f3fad709e21-Abstract.html (Accessed: June 16, 2026).

Pedregosa, F. *et al.* (no date) “Scikit-learn: Machine Learning in Python,” *MACHINE LEARNING IN PYTHON* [Preprint].

Prechelt, L. (1998) “Early Stopping - But When?,” in G.B. Orr and K.-R. Müller (eds.) *Neural Networks: Tricks of the Trade*. Berlin, Heidelberg: Springer, pp. 55–69. Available at: https://doi.org/10.1007/3-540-49430-8_3.

Srivastava, N. *et al.* (2014) “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, 15(56), pp. 1929–1958.